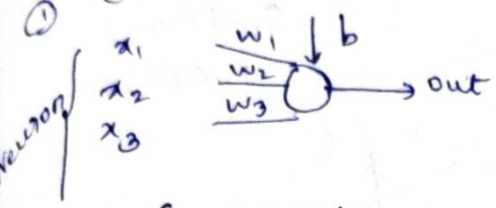
Summary :-

① Create a Neural Network.



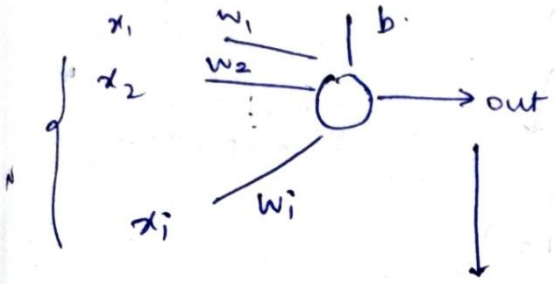
$out = \vec{w} \cdot \vec{x} + b$

$\vec{w} = (w_1, w_2, w_3)$

$\vec{x} = (x_1, x_2, x_3)$

✓ $(w_1 x_1 + x_2 w_2 + x_3 w_3 + b)$

$out \Rightarrow \left(\sum_{i=1}^{N} w_i \cdot x_i + b. \right)$ N ⇒ Number of input



$\left(out = \sum_{i=1}^{N} x_i w_i + b \right)$

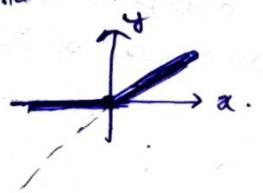
f^{xn} of a MAC-unit

Multiply and accumilate unit

Activation → neuron_output

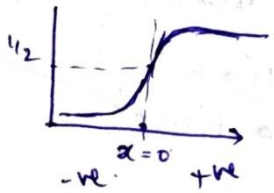
ReLU, sigmoid

$\max(0, x) = y$

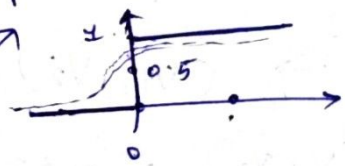


Rectified Linear unit

$\sigma(x) = \frac{1}{1+e^{-x}}$.

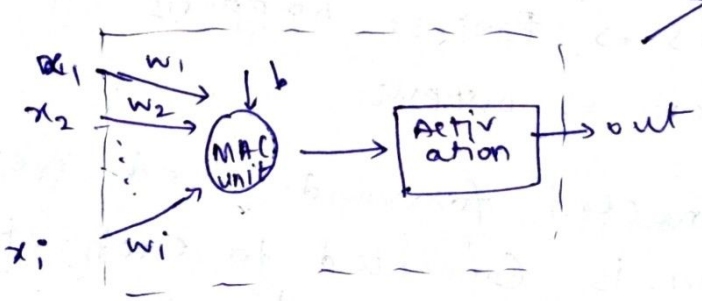


more smoothed step f^{xn}



↑ step

each neuron;



each neuron.

Multiply each $w_1 x_1$

acc ⇒ accumilate

final ⇒

$\left(\sum_{i=1}^{N} w_i x_i + b \right)$

given to Activation f^{xn}, Called as firing of Neuron.

acc = 0

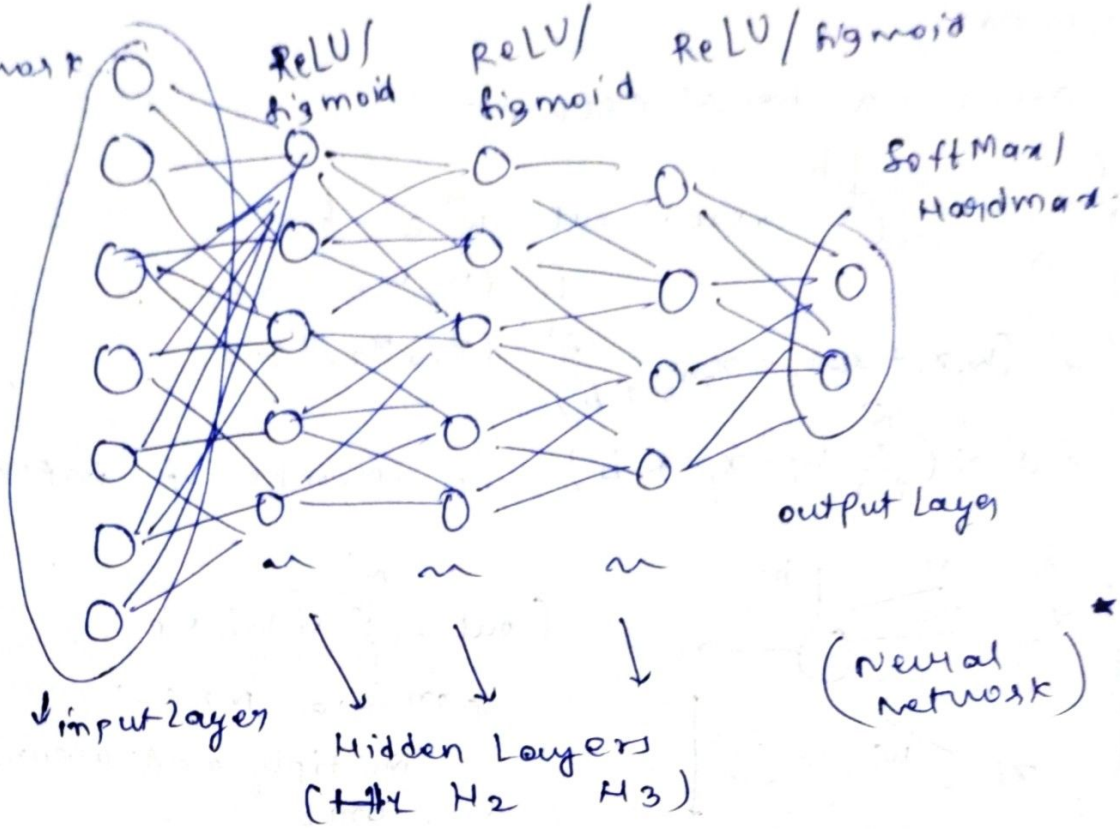
acc = $w_1 x_1$

acc = $w_1 x_1 + w_2 x_2$

acc = $w_1 x_1 + w_2 x_2 + w_3 x_3$

$\left(acc = \sum_{i=1}^{N} w_i x_i \right) + bias$

② Network



ReLU → preffered in Latest NN.

SoftMax → (V1, V2) ⇒ Convert into probabilities add upto 1.
(P1, P2) P1+P2=1.

HardMax → (V1, V2) max(P1, P2) ⇒ output High once at a time.

max (V1, V2) ⇒ output High once at a time.

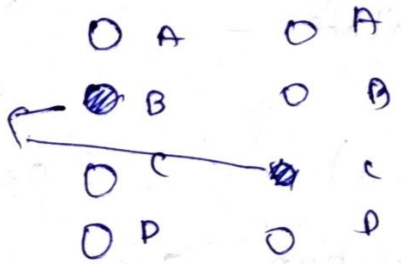
Hidden Layers ⇒ Process happens.

output Layer ⇒ output.

Fully Connected forward feed Network ⇒

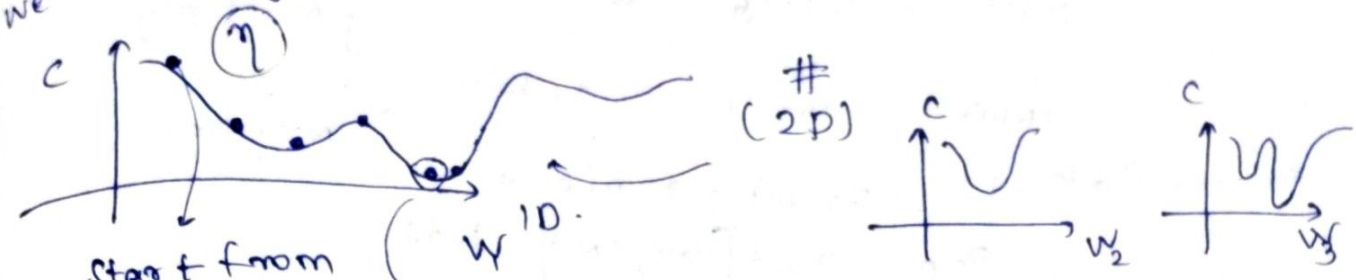
→ Every Neuron is Connected to each other on Adjacent layers.
→ only value goes forward..
→ output Hardmax/ SoftMax (High only one at a time)

used to find the output.



⇒ How we change weight / biases depend on Backpropagation.

we use gradient descent Method.



Start from Random point

C ⇒ Cost function.
↓ tells us how deviated the defined output from the Actual output
$C(W_1, W_2 \cdots W_N, b)$.

$$W_i' = W_i - \eta\left(\frac{\partial c}{\partial w_i}\right)$$

next point Start ↓ Learning Rate

(min).

$$\left[\begin{array}{l} w_i' = w_i - \eta\left(\frac{\partial c}{\partial w_i}\right) \\ (i=1, \rightarrow N) \end{array}\right] \star$$

$$\& \; b' = b - \eta\left(\frac{\partial c}{\partial b}\right) \star$$

→ find the value of W_i & b for which (C) is minimized
⇓ MSE (mean square error).

N → Large, it is not feasible.

Rathes than Large N; we will take some K such that Value's are Approximately same.

⇒ All thing done by software python.

epoch, → one epoch is the amount of data, taken Random values and Trained until the Accuracy / Model is exhausted.
→ Saturated.

③ In our Neuron design, i tried Combinational one;

Use FP 1.15 ⇒ 16 bit Q1.15

fixed point signed 16 bit.

$$\underbrace{Q1.15}_{16\,bit} \times \underbrace{Q1.15}_{16\,bit} = \underbrace{Q\,2.30}_{32\,bit}$$ } Multiply N times

Accumilate ⇒ 32 bit + 32 bit + ...

Rep $32 + \lceil \log_2(N) \rceil$ bits.

≥ integer.

Use Clocked Architecture

P ⇒ Parallelism; Number of Multiplications done in one Clock Cycle

More the "p", More the Speed.

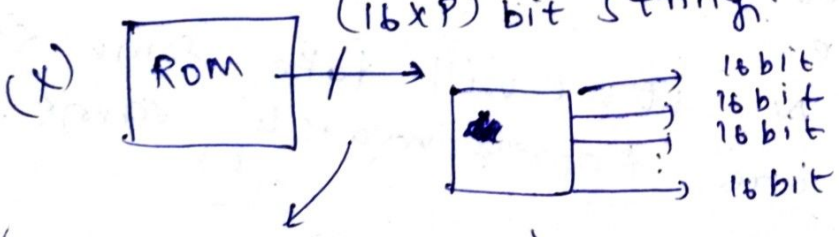
DSP's perform Multiplication

$(P)_{max}$ = Number of DSP's

but ↑ P, will ↑ area on "FPGA".

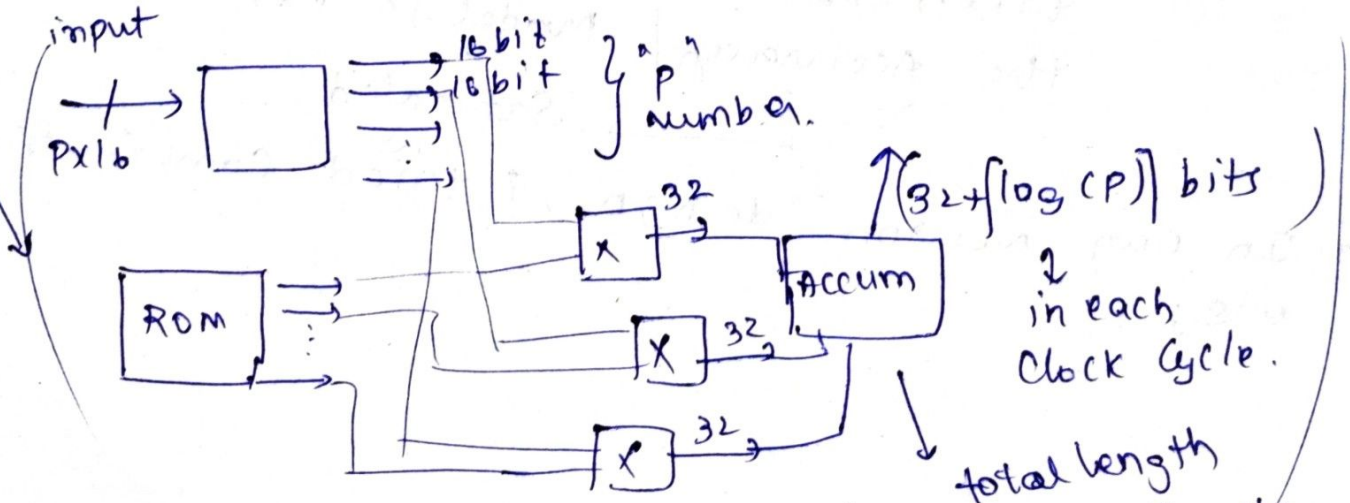
weight = 16 bit each

(16xP) bit string.



Number of string length ↑.

There Req extra hardware (not Much).



for "N" total diff inputs.

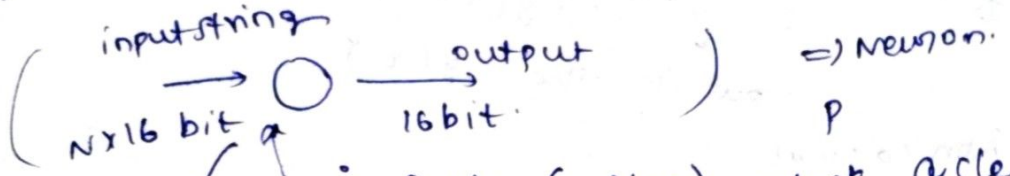
we Rep ⇒ (N/p) clock cycles

$$+\left((N\%p) \;?\; 0 : 1\right)$$

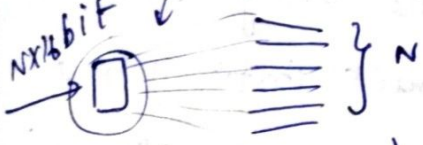
$==0$

So; Rep number of clock cycles.

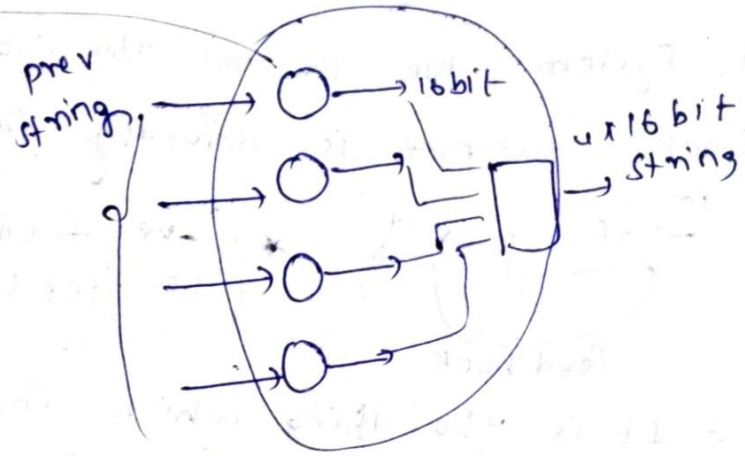
but we have to provide p of 16 bit



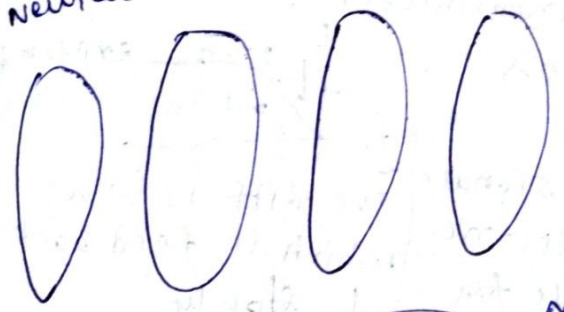
in cycles $(1 + N/p)$ clock cycles



(generate wires) simple.



Neural network

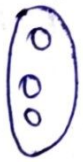


input layer (Just input's).

ReLU / Sigmoid.

Multiple Hidden Layers

Hidden Layer.



→ (output layer other type)

~~Neuronetwork~~ of Activation fxn

⇒ 28 × 28 pixel

784